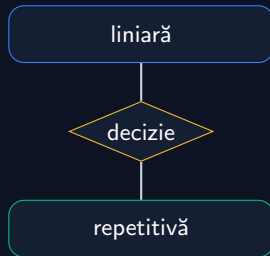


Introducere în pseudocod

Bazele Informaticii

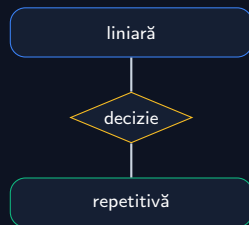


Ce este pseudocodul?

PRIMII PAȘI

Pseudocodul este un limbaj folosit pentru a descrie pașii unui algoritm.

- ▶ Împrumută structuri din limbaje de programare reale (Pascal, C/C++, BASIC etc.), dar este gândit pentru a fi citit de oameni, nu executat de calculator.
- ▶ Detaliile de implementare sunt de obicei omise — cum ar fi declararea variabilelor sau sintaxa specifică unui limbaj anume.
- ▶ Pseudocodul poate combina expresii în limbaj natural cu notații matematice compacte care nu există în limbajele de programare reale.



Concepte de bază

PRIMII PAȘI

Conceptele întâlnite în algoritmii scriși în pseudocod sunt similare celor din limbajele de programare reale:

1

Instrucțiuni

2

Variabile

3

Constante

4

Operații

5

Expresii

Există mai multe variante de pseudocod. Acest material folosește varianta întâlnită la problemele de Informatică de la Bacalaureat — aceeași predată la orele de algoritmică din liceu.

Variabile și constante

ELEMENTE DE BAZĂ

Variabilele nu se declară niciodată.

Numele lor respectă regulile obișnuite pentru identificatori: litere, cifre și caracterul underscore — iar un nume nu poate începe niciodată cu o cifră.

Constantele sunt valori literale:

- ▶ numere întregi și reale
- ▶ caractere delimitate cu apostrof (')
- ▶ șiruri de caractere delimitate cu apostrof (') sau ghilimele (")

Constantele matematice, precum π , pot apărea și ele direct în expresii.

```
// exemple
```

```
varsta ← 15    // intreg
```

```
medie ← 3.85  // real
```

```
nume ← 'Alex' // sir de  
                caractere
```

```
nota ← 'A'    // caracter
```

Operații cu date

ELEMENTE DE BAZĂ

Operații aritmetice

Adunare: $a + b$ — întreg sau real, în funcție de context

Scădere: $a - b$

Înmulțire: $a * b$

Împărțire: a/b — întotdeauna real; partea întreagă a câtului: $[a/b]$

Rest: $a \% b$ — doar pentru numere întregi, rezultat întreg

Unele variante folosesc DIV și MOD în loc de [] și %. Pot apărea și ridicarea la putere sau radicalul.

Operații relaționale și logice

Relaționale:

$=, \neq, <, \leq, >, \geq$

Operanzii sunt de obicei numere; rezultatul este o valoare booleană (**adevărat** sau **fals**).

Operații logice:

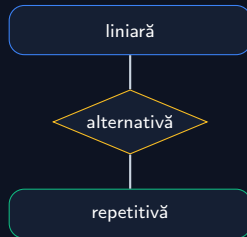
NU, ȘI, SAU — combină sau inversează valori booleene.

Instrucțiuni: Noțiuni de bază

ELEMENTE DE BAZĂ

Instrucțiunile sunt părțile unui algoritm care au efectiv un efect — modifică valori de variabile, citesc sau afișează date, repetă acțiuni și așa mai departe.

- ▶ De obicei fiecare instrucțiune se scrie pe rândul ei (sau pe mai multe, pentru cele complexe).
- ▶ Pentru a economisi spațiu, două sau mai multe instrucțiuni simple pot fi scrise pe același rând, separate prin ;.
- ▶ Orice algoritm poate fi construit din trei tipuri de structuri:
 - ▶ **liniară**
 - ▶ **alternativă** (decizie)
 - ▶ **repetitivă**



Citare, Scriere și Atribuire

Cele trei instrucțiuni cu care începe orice algoritm în pseudocod.

Citirea datelor

DEMO 1

`citește <listă de variabile>`

`<listă de variabile>` este o succesiune de variabile separate prin virgulă.

- ▶ Valorile succesive sunt preluate de la tastatură, în ordine, și stocate în variabilele din listă.
- ▶ O instrucțiune `citește` este de obicei însoțită de precizări despre datele așteptate — tipul lor, domeniul de valori valide și așa mai departe.

```
citeste a, b, c
```

Scrierea datelor

DEMO 2

`scrie <listă de expresii>`

`<listă de expresii>` este o succesiune de expresii separate prin virgulă.

- Expresiile sunt evaluate în ordine, iar rezultatele lor sunt afișate pe ecran.

```
scrie 'a + b = ', a + b
```

```
// daca a = 3 si b = 4:
```

```
// a + b = 7
```

Instrucțiunea de atribuire

DEMO 3

$\langle \text{variabilă} \rangle \leftarrow \langle \text{expresie} \rangle$

Expresia este evaluată mai întâi, iar valoarea rezultată este apoi stocată în variabilă.

- ▶ Aceasta este instrucțiunea de bază din spatele oricărui calcul dintr-un algoritm scris în pseudocod.

```
a ← 0  
S ← a + b  
i ← i + 1
```

Structura alternativă

LUAREA DECIZIILOR

Ordinea de execuție:

1. Se evaluează <condiția>.
2. Dacă este adevărată, se execută <instrucțiuni1>, apoi se trece la instrucțiunea următoare.
3. Dacă este falsă, se execută <instrucțiuni2> — atunci când există o ramură altfel — apoi se trece la instrucțiunea următoare.

```
┌  daca <conditie> atunci  
|    <instrucțiuni1>  
|  altfel  
|    <instrucțiuni2>
```



```
┌  daca <conditie> atunci  
|    <instrucțiuni1>
```



Structuri repetitive

Trei moduri de a repeta o succesiune de instrucțiuni.

**PENTRU ...
EXECUTĂ**

**CÂT TIMP
...
EXECUTĂ**

**REPETĂ...
PÂNĂ CÂND**

Clasificarea structurilor repetitive

TAXONOMIE ȘI CRITERII

După poziția testului

1. Test inițial (precondiție)

- ▶ Testat *înainte* de corpul buclei
- ▶ Execuții: 0 sau mai multe ori
- ▶ **CÂT TIMP ... EXECUTĂ**
- ▶ **PENTRU ... EXECUTĂ**

2. Test final (postcondiție)

- ▶ Testat *după* corpul buclei
- ▶ Execuții: cel puțin o dată
- ▶ **REPETĂ...PÂNĂ CÂND**

După numărul de repetări

1. Număr cunoscut de pași

- ▶ Determinat dinainte (mărginit)
- ▶ **PENTRU... EXECUTĂ**

2. Număr necunoscut de pași

- ▶ Depinde de o condiție (nemărginit)
- ▶ **CÂT TIMP ... EXECUTĂ**
- ▶ **REPETĂ...PÂNĂ CÂND**

Structura PENTRU

STRUCTURI REPETITIVE — PAȘI CUNOSCUȚI

```
┌ pentru <var> ← <inceput>, <sfarsit>,  
  <pas> executa  
|   <instructiuni>  
└■
```

1. *inceput*, *sfârșit* și *pas* se evaluează la numere întregi;
2. *pas* este 1 sau -1 , implicit 1 atunci când e omis.
3. Dacă $\text{pas} = 1$ și $\text{inceput} > \text{sfârșit}$, instrucțiunile nu se execută niciodată.

Demo — suma și produsul primelor n numere

```
S ← 0; P ← 1  
┌ pentru i ← 1, n executa  
|   S ← S + i  
|   P ← P * i  
└■  
scrie S, ' ', P
```

Structura CÂT TIMP

STRUCTURI REPETITIVE — TEST INIȚIAL

```
┌ cat timp <conditie> executa  
|   <instructiuni>  
└ ─■
```

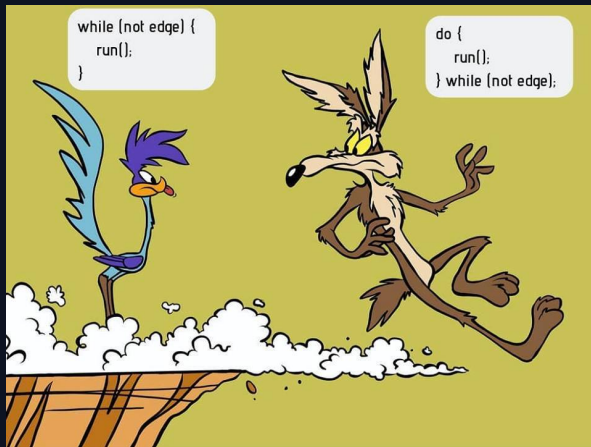
1. Se evaluează <condiția>.
2. Dacă este adevărată, se execută instrucțiunile și se revine la pasul 1.
3. Dacă este falsă, se trece la instrucțiunea următoare.

Demo — suma cifrelor unui număr

```
citeste n  
S ← 0  
┌ cat timp n ≠ 0 executa  
|   S ← S + n % 10  
|   n ← [n / 10]  
└ ─■  
scrie S
```

STRUCTURA REPETĂ...PÂNĂ CÂND

CÂT TIMP VS. REPETĂ...PÂNĂ CÂND



Structura REPETĂ...PÂNĂ CÂND

STRUCTURI REPETITIVE — TEST FINAL

```
┌ repeta  
|   <instructiuni>  
└ pana cand <conditie>
```

1. Se execută instrucțiunile.
2. Se evaluează <condiția>.
3. Dacă este falsă, se revine la pasul 1. Dacă este adevărată, se trece mai departe.

Important: Chiar dacă de la **început** condiția este **adevărată**, instrucțiunile **SE EXECUTĂ cel puțin o dată**.

Demo — numărul de cifre ale unui număr

```
citeste n  
cnt ← 0  
┌ repeta  
|   cnt ← cnt + 1  
|   n ← [n / 10]  
└ pana cand n = 0  
scrie cnt
```

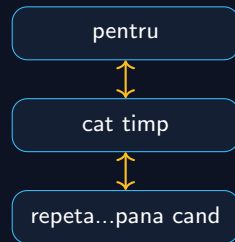
Echivalența structurilor repetitive

STRUCTURI REPETITIVE

Multe exerciții vă dau un algoritm și vă cer unul echivalent care folosește o altă structură repetitivă.

Doi algoritmi sunt echivalenți dacă, pentru orice set de date de intrare valide, produc exact aceleași rezultate.

Următoarele slide-uri parcurg regulile standard de transformare între structurile PENTRU, CÂT TIMP și EXECUTĂ...CÂT TIMP.



PENTRU $\rightarrow$ CÂT TIMP

ECHIVALENȚA STRUCTURILOR

Înainte

```
┌ pentru  $i \leftarrow a, b$  executa  
│   <instructiuni>  
└ ─■
```

Echivalent

```
   $i \leftarrow a$   
┌ cat timp  $i \leq b$  executa  
│   <instructiuni>  
│    $i \leftarrow i + 1$   
└ ─■
```

PENTRU → REPETĂ...PÂNĂ CÂND

ECHIVALENȚA STRUCTURILOR

Înainte

```
┌ pentru i ← a, b executa  
│   <instructiuni>  
└ ─■
```

Echivalent

```
i ← a  
┌ daca i ≤ b atunci  
│ ┌ repeta  
│ │   <instructiuni>  
│ │   i ← i + 1  
│ └ pana cand i > b  
└ ─■
```

CÂT TIMP → REPETĂ...PÂNĂ CÂND

ECHIVALENȚA STRUCTURILOR

Înainte

```
┌ cat timp <conditie> executa  
|   <instructiuni>  
└ ■
```

Echivalent

```
┌ daca <conditie> atunci  
| ┌ repeta  
| |   <instructiuni>  
| └ pana cand NU (<conditie>)  
└ ■
```

Întrebări?

Urmează: ora de laborator!! Introducere în Python și VS Code.

Vă mulțumesc pentru atenție!